Data Analysis, Machine Learning and Deep Learning in R

A Comprehensive Guide

Vo Hoang Nhat Khang

Date: October 15, 2024

Contents

1 Welcome to R **3**
1.1 Introduction . . . 3
1.2 Getting Started with RStudio . . . 4
1.2.1 RStudio Interface Components . . . 4
1.2.2 Basic Syntax in R . . . 4
1.2.3 Writing Your First Lines of Code . . . 5
1.3 Data Manipulation with R . . . 8
1.3.1 Reading Data . . . 8
1.3.2 Data Manipulation with `dplyr` . . . 9
1.4 Data Visualization with ggplot2 . . . 13
1.4.1 Creating a Scatter Plot . . . 13
1.4.2 Creating a Bar Plot . . . 14
1.4.3 Creating a Heatmap . . . 15

2 Machine Learning with R **17**
2.1 Data Manipulation . . . 17
2.1.1 Loading and Cleaning Data . . . 17
2.1.2 Exploratory Data Analysis (EDA) . . . 18
2.1.3 Principal Component Analysis (PCA) . . . 22
2.2 Statistical Modeling . . . 25
2.2.1 Linear Regression . . . 25
2.2.2 Logistic Regression . . . 26
2.3 Machine Learning . . . 27
2.3.1 K-Nearest Neighbors (KNN) . . . 27
2.3.2 Random Forest . . . 31
2.4 Model Evaluation . . . 33
2.4.1 Confusion Matrix and Accuracy . . . 33
2.4.2 ROC Curve . . . 34

3 Deep Learning Features in R **37**
3.1 Popular Packages for Deep Learning . . . 37
3.2 Building Neural Networks with Keras . . . 37
3.2.1 Loading and Preprocessing Data . . . 37
3.2.2 Defining the Model . . . 38
3.2.3 Compiling the Model . . . 39
3.3 Training Deep Learning Models . . . 39
3.4 Using Pretrained Models with the CIFAR Dataset . . . 42
3.4.1 Load and Preprocess the CIFAR-10 Dataset . . . 42

3.4.2 Load and Modify the Pretrained ResNet50 Model 42
3.4.3 Add Custom Layers for Fine-Tuning 43
3.4.4 Compile the Model 43
3.4.5 Train the Model 44
3.4.6 Evaluate the Model on Test Data 44

Chapter 1

Welcome to R

1.1 Introduction

R is a powerful programming language for statistical computing and graphics. It offers numerous packages and tools for data analysis, visualization, and machine learning.

To get started with R, follow these steps:

1. Download and install R from `https://cran.r-project.org/`
2. Download and install RStudio from `https://www.rstudio.com/products/rstudio/download/`

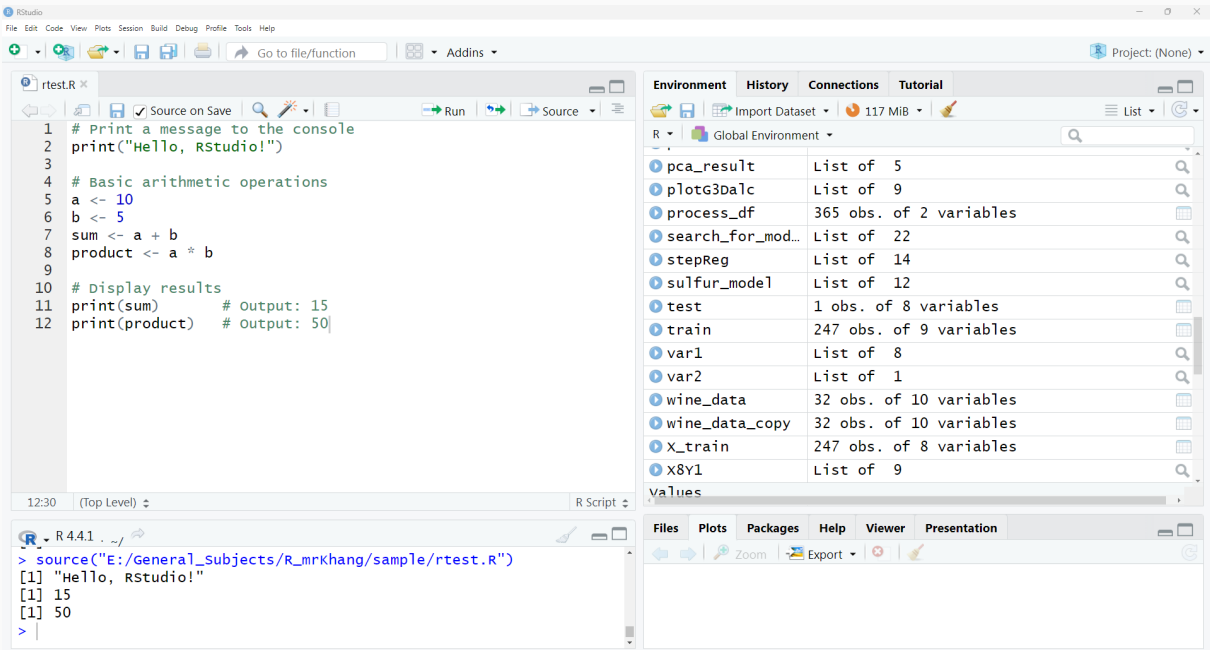

Figure 1.1: RStudio UI

RStudio provides a user-friendly interface for working with R.

1.2 Getting Started with RStudio

RStudio is an Integrated Development Environment (IDE) for R that provides a user-friendly interface to help manage and execute R code efficiently. Here, we will explore the key components of the RStudio interface and write our first lines of code.

1.2.1 RStudio Interface Components

The RStudio interface consists of several key panels:

- **Source Panel**: This panel is where you can write, edit, and save your R scripts (.R files). You can also create and execute RMarkdown (.Rmd) files for documentation.
- **Console Panel**: This is where you execute R commands and immediately see the output. You can type commands directly here or send them from the Source panel.
- **Environment/History Panel**: The Environment tab shows all the variables, data frames, and functions currently loaded in your session, while the History tab provides a record of all the commands you have executed.
- **Files/Plots/Packages/Help Panel**: This panel has multiple tabs:
  - `Files`: Displays the directory contents, allowing you to navigate and manage files.
  - `Plots`: Displays graphical output from plots created with R's plotting functions.
  - `Packages`: Manages R packages. You can install, update, or load packages from here.
  - `Help`: Provides access to documentation for R functions and packages. You can search for help using `?function_name` in the Console.

1.2.2 Basic Syntax in R

Assignment Operators

In R, you can assign values to variables using either the assignment operator `<-` or the equal sign `=`. These operators allow you to store data and reuse it throughout your code.

- Example:

```
x <- 10    # Using the assignment operator
y = 20     # Using the equal sign
```

Comments

Comments are crucial for making your code more understandable. In R, any text following the `#` symbol on the same line is considered a comment and will be ignored during execution.

- Example:

```
# This is a comment
z <- x + y  # Adds x and y, assigns result to z
```

Functions

Functions in R are predefined operations or procedures that perform specific tasks. You can call a function by using its name followed by parentheses `()` containing any arguments the function requires.

- Example:

```
result <- sum(5, 10)  # Calculates the sum of 5 and 10
```

Vectors

Vectors are one-dimensional arrays that can hold multiple elements of the same type, such as numbers or characters. You can create a vector using the `c()` function, which combines elements into a vector.

- Example:

```
numbers <- c(1, 2, 3, 4, 5)  # Creates a numeric vector
```

1.2.3 Writing Your First Lines of Code

Let's start by writing some basic R code. Enter the following commands in the Console or Source panel:

```
# Print a message to the console
print("Hello, RStudio!")

# Basic arithmetic operations
a <- 10
b <- 5
sum <- a + b
product <- a * b

# Display results
print(sum)        # Output: 15
print(product)    # Output: 50
```

In this code, we use the `print` function to display a message and perform basic arithmetic operations.

Figure 1.2: Output

Data Types

R has several built-in data types that are fundamental for data manipulation and analysis.

- **Numeric:** Represents real numbers, including integers and decimals.
- **Integer:** A specific numeric type for whole numbers. You can specify integers by appending `L` to the number.
- **Character:** Represents strings of text. Strings are created using either single or double quotes.
- **Logical:** Represents boolean values, either `TRUE` or `FALSE`.
- **Factor:** Used to represent categorical data with fixed levels.

```r
num_var <- 3.14             # Numeric
int_var <- 42L               # Integer
char_var <- "Hello, R!"      # Character
log_var <- TRUE               # Logical
```

Control Structures

Control structures in R allow you to control the flow of execution based on conditions.

- **If-Else Statement:** Executes a block of code based on a condition.

```r
age <- 18
if (age >= 18) {
    print("You are an adult.")
} else {
    print("You are a minor.")
}
```

- **For Loop:** Repeats a block of code a specified number of times.

```r
for (i in 1:5) {
    print(paste("Iteration:", i))
}
```

- **While Loop:** Repeats a block of code while a specified condition is `TRUE`.

```r
count <- 1
while (count <= 5) {
    print(count)
    count <- count + 1
}
```

Basic Input and Output

R provides various functions to read data from various sources and display output.

- **Reading Data:** You can read data from files using functions such as `read.csv()` and `read.table()`.

```
data <- read.csv("datafile.csv")  # Read data from a CSV file
```

- **Displaying Output:** Besides `print()`, you can use `cat()` for more formatted output.

```
cat("Hello, R!\n")  # Displays the message without quotes
```

Basic Data Structures

R has several built-in data structures that are essential for storing and manipulating data:

- **Lists:** A versatile structure that can hold different types of elements.

```
my_list <- list(name = "Alice", age = 25, height = 5.5)
```

- **Data Frames:** A table-like structure where each column can contain different types of data. It is similar to a spreadsheet.

```
df <- data.frame(name = c("Alice", "Bob"), age = c(25, 30))
```

- **Matrices:** A two-dimensional array that can hold elements of the same type.

```
my_matrix <- matrix(1:6, nrow = 2, ncol = 3)  # 2 rows, 3 columns
```

Using Libraries

R has a rich ecosystem of packages that extend its functionality. You can install and load packages using the following commands:

- `install.packages("package_name")` installs a package.
- `library(package_name)` loads a package for use.

```
install.packages("ggplot2")  # Install ggplot2 for data visualization
library(ggplot2)               # Load ggplot2
```

Exercises: Try these exercises to practice the basic syntax elements in R:

1. **Variable Assignment:** Create a variable named `age` and assign it your current age. Then, create a variable named `next_year` that adds 1 to the `age` variable. Print the value of `next_year`.

```
# Solution
age <- 25
next_year <- age + 1
print(next_year)
```

2. **Comments:** Write a line of code that creates a variable named `temperature` and assigns it a value of 22. Use a comment to explain what the variable represents.

```r
# Solution
temperature <- 22  # Temperature in degrees Celsius
```

3. **Using Functions:** Use the `mean()` function to calculate the average of the numbers 10, 20, and 30. Store the result in a variable named `average` and print it.

```r
# Solution
average <- mean(c(10, 20, 30))
print(average)
```

4. **Creating Vectors:** Create a vector named `heights` that contains the values 170, 165, 180, and 175. Use the `length()` function to find the number of elements in the vector and print the result.

```r
# Solution
heights <- c(170, 165, 180, 175)
print(length(heights))
```

5. **Exploring Functions:** Look up the `summary()` function in R, which provides a summary of its input. Create a vector named `scores` with values 85, 90, 78, 88, and 95, and then use the `summary()` function on this vector. Print the output.

```r
# Solution
scores <- c(85, 90, 78, 88, 95)
print(summary(scores))
```

1.3 Data Manipulation with R

1.3.1 Reading Data

R can read data from various sources, such as CSV and Excel files.

Reading a CSV File

Use the `read.csv()` function to read data from a CSV file:

```r
# Reading a CSV file
data <- read.csv("path/to/your/file.csv", header = TRUE, sep = ",")
head(data)  # View the first few rows
summary(data)  # Summary statistics of the data
```

Parameters include:

- `file`: Specifies the path to the CSV file.
- `header`: A logical value. If `TRUE`, the first row is treated as the column headers. Defaults to `TRUE`.

- `sep`: The field separator character. The default is a comma (`,`) for CSV files.

The `head()` function provides a quick preview of the dataset, while `summary()` gives an overview of each column, including min, max, median, and mean values.

Reading an Excel File

To read Excel files, you'll need the `readxl` package, which provides the `read_excel()` function:

```r
# Install and load the readxl package if not already installed
install.packages("readxl")  # Run this line once to install
library(readxl)

# Reading an Excel file
data <- read_excel("path/to/your/file.xlsx", sheet = 1, col_names = TRUE)
head(data)  # View the first few rows
summary(data)  # Summary statistics of the data
```

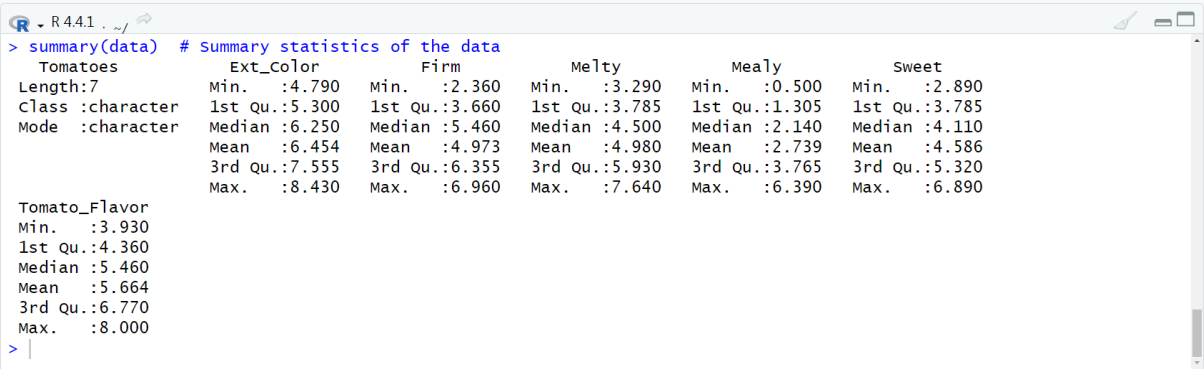

Figure 1.3: Output

Parameters include:

- `path`: Specifies the path to the Excel file.
- `sheet`: The sheet number or name to read from. Defaults to the first sheet.
- `col_names`: A logical value. If `TRUE`, the first row is treated as the column headers. Defaults to `TRUE`.

1.3.2 Data Manipulation with `dplyr`

The `dplyr` package is a powerful tool for data manipulation in R, enabling you to filter, select, mutate, and summarize data efficiently. The `%>%` operator, known as the "pipe" operator, allows chaining multiple operations together in a clear and readable format.

To install and import the `dplyr` package, use the following commands:

```r
# Install dplyr package (run this once)
install.packages("dplyr")

# Load dplyr package
library(dplyr)
```

Filtering Data

The `filter()` function allows you to subset rows based on conditions. For example:

```r
# Filter rows where column1 is greater than 5
filtered_data <- data %>%
  filter(Ext_Color > 5)
```

```
R 4.4.1 · ~/
> print(filtered_data)
# A tibble: 5 × 7
  Tomatoes Ext_Color  Firm Melty Mealy Sweet Tomato_Flavor
  <chr>        <dbl> <dbl> <dbl> <dbl> <dbl>         <dbl>
1 A             8.43  5.46  4.5   0.5   6.89          8
2 C             6.79  6.96  3.29  1.61  4.11          5.46
3 D             6.25  6.39  3.96  5.07  2.89          3.93
4 F             8.32  4.93  4.79  1     6.46          7.29
5 G             5.71  2.36  7.64  2.14  3.61          4.43
>
```

Figure 1.4: Output

In this example, only rows with `column1` values greater than 5 are kept.

Selecting Columns

Use the `select()` function to choose specific columns. This is particularly useful for narrowing down datasets with many columns.

```r
# Select columns column1 and column2
selected_data <- data %>%
  select(Ext_Color, Firm)
```

```
R 4.4.1 · ~/
  Ext_Color  Firm
      <dbl> <dbl>
1      8.43  5.46
2      4.79  2.39
3      6.79  6.96
4      6.25  6.39
5      4.89  6.32
6      8.32  4.93
7      5.71  2.36
>
```

Figure 1.5: Output

Creating New Columns with `mutate()`

The `mutate()` function allows you to create new variables or modify existing ones.

```
# Create a new column that is twice the value of column1
mutated_data <- data %>%
  mutate(new_column = Ext_Color * 2)
```

In this example, `new_column` is added, containing values that are twice those in `Ext_Color`.

```
R 4.4.1 · ~/
  Tomatoes Ext_Color  Firm Melty Mealy Sweet Tomato_Flavor new_column
  <chr>        <dbl> <dbl> <dbl> <dbl> <dbl>        <dbl>      <dbl>
1 A             8.43  5.46  4.5   0.5   6.89         8         16.9
2 B             4.79  2.39  7.07  6.39  3.96         4.29       9.58
3 C             6.79  6.96  3.29  1.61  4.11         5.46      13.6
4 D             6.25  6.39  3.96  5.07  2.89         3.93      12.5
5 E             4.89  6.32  3.61  2.46  4.18         6.25       9.78
6 F             8.32  4.93  4.79  1     6.46         7.29      16.6
7 G             5.71  2.36  7.64  2.14  3.61         4.43      11.4
>
```

Figure 1.6: Output

Grouping and Summarizing Data

Grouping and summarizing data are essential for exploratory data analysis. The `group_by()` and `summarise()` functions enable you to perform these tasks.

```
# Group by column2 and calculate the mean of column1 for each group
summarised_data <- data %>%
  group_by(column2) %>%
  summarise(mean_value = mean(column1))
```

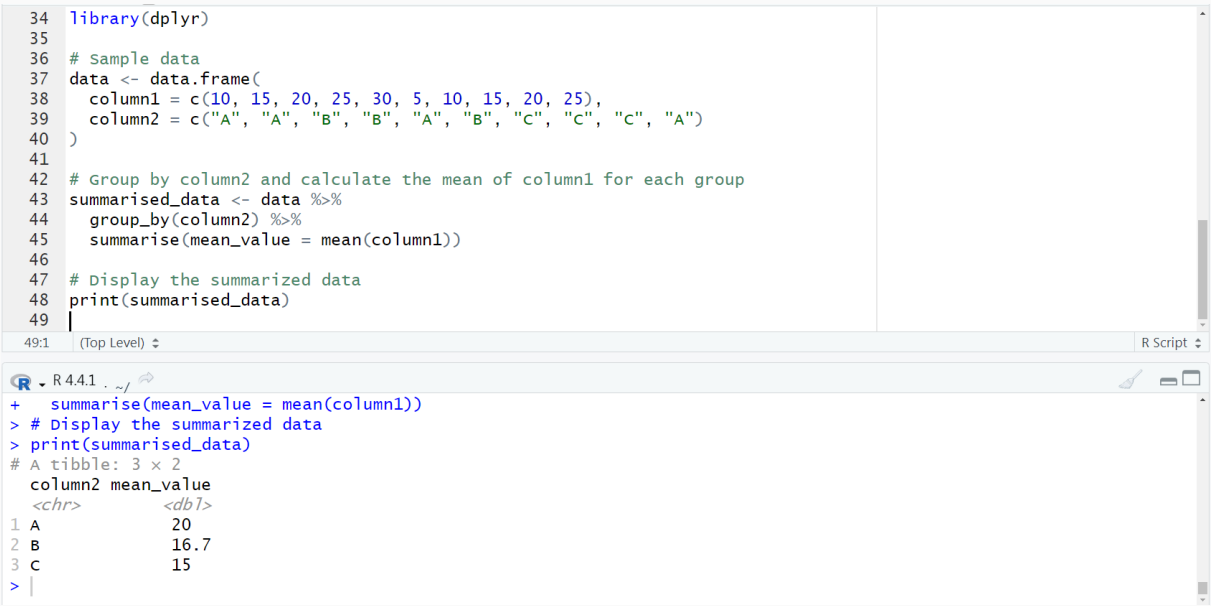

Figure 1.7: An example

Combining Operations with the Pipe Operator (%>%)

The true power of `dplyr` lies in combining these functions with the pipe operator. Here's an example that integrates filtering, selecting, mutating, and summarizing operations:

```
# Combined operations: filter, select, mutate, and summarise
result <- data %>%
  filter(column1 > 5) %>%
  select(column1, column2) %>%
  mutate(new_column = column1 * 2) %>%
  group_by(column2) %>%
  summarise(mean_value = mean(new_column))
```

In this pipeline, rows with `column1` greater than 5 are filtered, `column1` and `column2` are selected, a new column is created, and the mean of `new_column` is calculated for each group defined by `column2`.

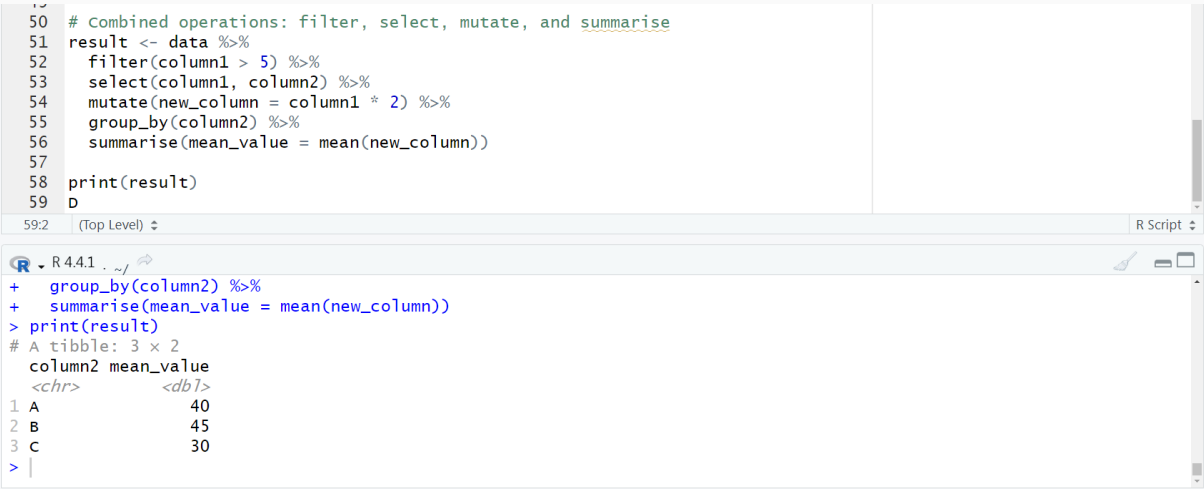

Figure 1.8: Output

To get familiar with `dplyr`, try the following exercise:

- Filter rows where a column value meets a certain condition.
- Select specific columns from the data.
- Create a new column using `mutate()` with a transformation.
- Group the data by one or more columns and summarize with a function of your choice (e.g., `mean()`, `sum()`).

Example:

```
# Exercise: Try to apply a different transformation and grouping on your data
your_data %>%
  filter(columnA < 10) %>%
  select(columnA, columnB, columnC) %>%
  mutate(new_metric = columnA / columnB) %>%
  group_by(columnC) %>%
  summarise(total = sum(new_metric))
```

Experiment with different conditions, columns, and functions to understand the versatility of `dplyr`.

1.4 Data Visualization with ggplot2

1.4.1 Creating a Scatter Plot

Visualizations are essential for data analysis.

```r
# Load ggplot2 package
library(ggplot2)

# Create a scatter plot
ggplot(data, aes(x = column1, y = column2)) +
  geom_point() +
  theme_minimal() +
  labs(title = "Scatter Plot", x = "X Axis", y = "Y Axis")
```

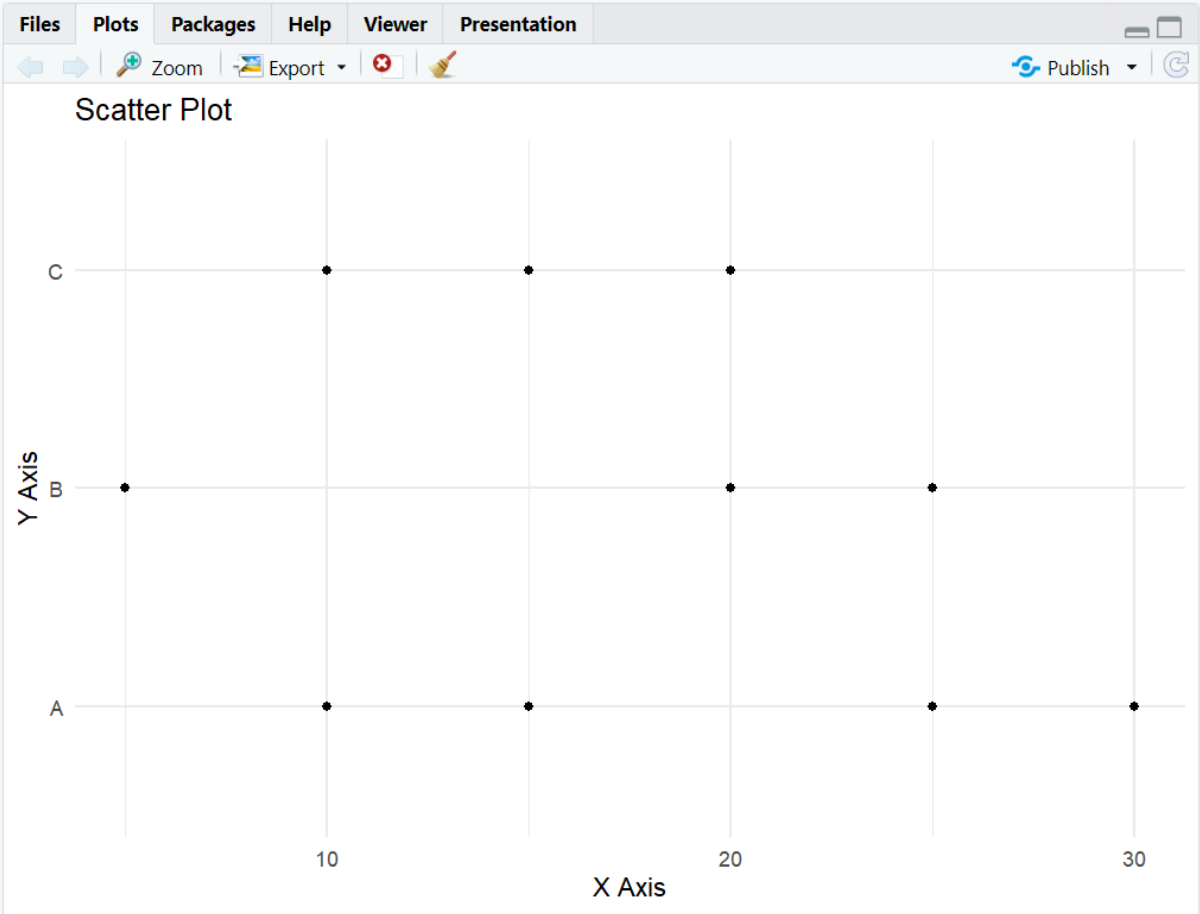

Figure 1.9: Output

The `ggplot2` package is flexible and enables creating various types of plots. This scatter plot shows the relationship between two variables, `column1` and `column2`.

1.4.2 Creating a Bar Plot

```r
# Create a bar plot
ggplot(data, aes(x = factor(column1), y = column2)) +
  geom_bar(stat = "identity") +
  theme_minimal() +
  labs(title = "Bar Plot", x = "Categories", y = "Values")
```

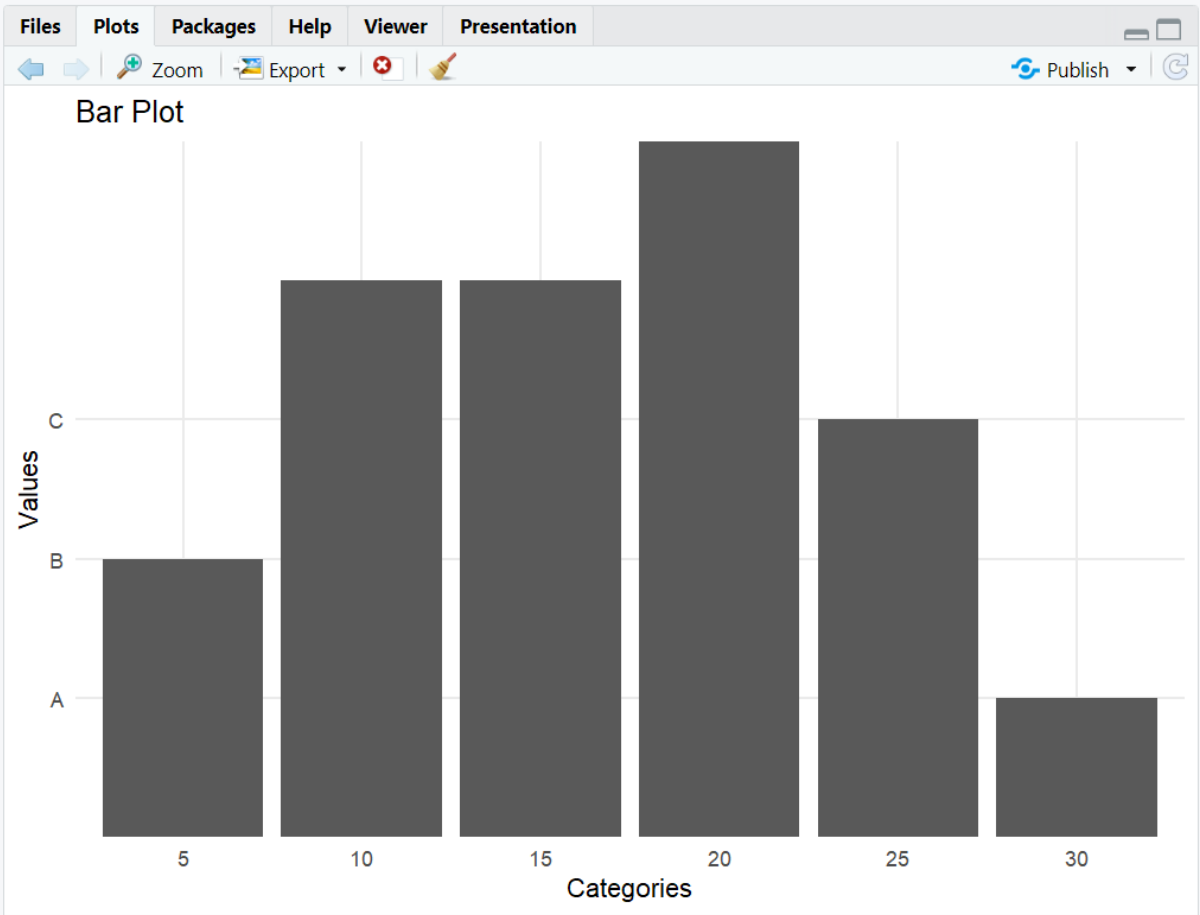

Figure 1.10: Output

The bar plot visualizes categorical data, with `column1` as categories and `column2` as values. The `geom_bar` function requires specifying `stat = "identity"` to plot actual values.

1.4.3 Creating a Heatmap

A heatmap is a graphical representation of data where individual values are represented by colors. It is especially useful for displaying correlations or patterns within data.

First, ensure you have the ggplot2 and reshape2 packages installed and loaded. The reshape2 package is useful for transforming data into a format suitable for ggplot2.

```
# Load the necessary libraries
library(ggplot2)
library(reshape2)  # For data transformation
```

After that, we create a correlation matrix and assign row names as well as column names:

```r
# Example data: a correlation matrix
data_matrix <- matrix(rnorm(100), nrow = 10)  # Create a 10x10 matrix of random numbers
rownames(data_matrix) <- paste("Row", 1:10)  # Assign row names
colnames(data_matrix) <- paste("Col", 1:10)  # Assign column names
```

Now we transform the matrix into a long format for `ggplot`:

```r
data_long <- melt(data_matrix)
colnames(data_long) <- c("X", "Y", "Value")
```

Use `ggplot` to create the heatmap by mapping `X` and `Y` to the two axes and `Value` to the color fill:

```r
# Create the heatmap with ggplot2
ggplot(data_long, aes(x = X, y = Y, fill = Value)) +
  geom_tile() +
  scale_fill_gradient(low = "white", high = "blue") +
  theme_minimal() +
  labs(title = "Heatmap Example", x = "Columns", y = "Rows")
```

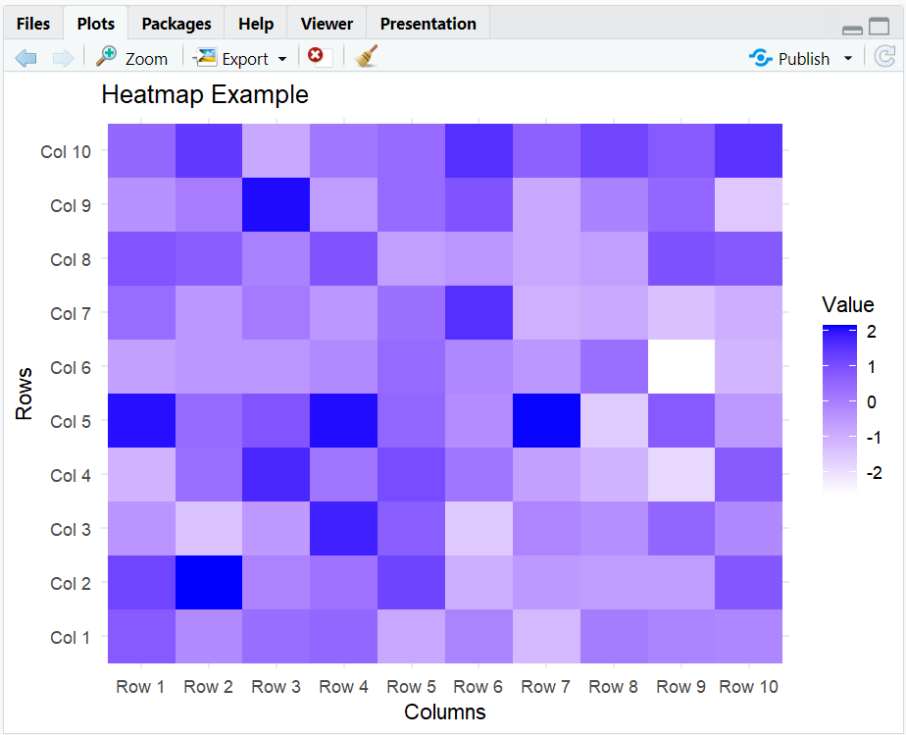

Figure 1.11: Output

Chapter 2

Machine Learning with R

2.1 Data Manipulation

2.1.1 Loading and Cleaning Data

In this section, first is to create a sample numerical dataset.

```
data <- data.frame(
  column1 = c(10, 15, 20, 25, 30, 5, 10, 15, 20, 25),
  column2 = c(20, 30, 55, 10, 2, 3, 4, 8, 12, 24)
)
```

After loading the data, we will clean it by filtering out rows with missing values and creating new columns as needed.

```
# Clean data
clean_data <- data %>%
  filter(!is.na(column_name)) %>%  # Remove rows where 'column_name' is NA
  mutate(new_column = column1 + column2)  # Create a new column as the sum of
  ↪  'column1' and 'column2'
```

rtest.R* clean_data

Filter

	column1	column2	new_column
1	10	20	30
2	15	30	45
3	20	55	75
4	25	10	35
5	30	2	32
6	5	3	8
7	10	4	14
8	15	8	23
9	20	12	32
10	25	24	49

Figure 2.1: Output

Now that we have loaded and cleaned our dataset, we can proceed to perform exploratory data analysis to gain insights into the data structure and relationships.

2.1.2 Exploratory Data Analysis (EDA)

In this section, we conduct a thorough exploratory data analysis (EDA) to understand the data's structure, distribution, and relationships. This includes generating summary statistics, visualizing data, detecting missing values, and analyzing correlations.

Summary Statistics and Grouping

Generating summary statistics provides an overview of the data, including measures of central tendency and dispersion. Grouping the data by specific categories allows us to identify patterns and trends across different groups.

```r
# Summary statistics
summary(clean_data)  # Get descriptive statistics for all columns in 'clean_data'

# Grouped summary
clean_data %>%
  group_by(category_column) %>%  # Group data by 'category_column'
  summarize(mean_value = mean(numeric_column, na.rm = TRUE))  # Calculate mean of 'numeric_column', ignoring NA values
```

```
> summary(clean_data)  # Get descriptive statistics for all columns in 'clean_data'
    column1         column2       new_column
 Min.   : 5.00   Min.   : 2.0   Min.   : 8.00
 1st Qu.:11.25   1st Qu.: 5.0   1st Qu.:24.75
 Median :17.50   Median :11.0   Median :32.00
 Mean   :17.50   Mean   :16.8   Mean   :34.30
 3rd Qu.:23.75   3rd Qu.:23.0   3rd Qu.:42.50
 Max.   :30.00   Max.   :55.0   Max.   :75.00
>
```

Figure 2.2: Output

Data Visualization

Visualizations such as histograms, boxplots, and scatter plots are essential for exploring data distributions and relationships between variables. They help in identifying patterns, trends, and potential outliers.

Histogram

A histogram provides a visual representation of the distribution of a numeric variable. It shows the frequency of data points falling within specified ranges (bins). In our example, we will create a histogram for a numeric column in the dataset.

```r
# Load ggplot2 library for visualization
library(ggplot2)  # Ensure ggplot2 is loaded

# Histogram for numeric column distribution
histogram <- ggplot(clean_data, aes(x = numeric_column)) +  # Specify the data and aesthetic mapping
  geom_histogram(
    binwidth = 5,  # Set the width of each bin to 5
    fill = "skyblue",  # Color the bars with sky blue
    color = "black"  # Outline the bars with black color
  ) +
  xlab("Numeric Column") +  # Label for the x-axis
  ylab("Frequency") +  # Label for the y-axis
  ggtitle("Histogram of Numeric Column")  # Title for the histogram
histogram # Display the histogram
```

The key parameters in the histogram include:

- `binwidth`: Determines the width of each bin; adjusting this can change the granularity of the histogram.
- `fill`: Specifies the color of the bars; this can be customized to improve visual appeal.
- `color`: Sets the color of the bar borders, enhancing clarity and distinction between bars.

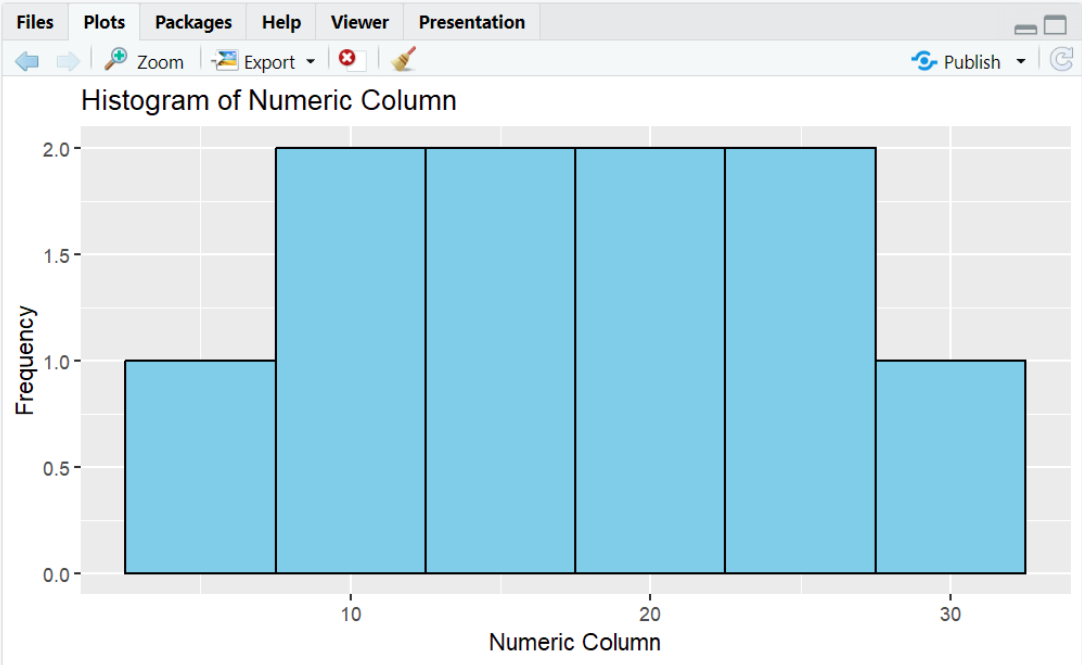

Figure 2.3: Output

Scatter Plot

A scatter plot is used to examine the relationship between two numeric variables. Each point represents an observation in the dataset, plotted according to the values of the two variables.

```r
# Scatter plot for relationship between two variables
scatter_plot <- ggplot(clean_data, aes(x = numeric_column1, y = numeric_column2)) +  # Specify the data and aesthetic mapping
  geom_point(alpha = 0.5) +  # Plot points with 50% transparency for better visibility
  xlab("Numeric Column 1") +  # Label for the x-axis
  ylab("Numeric Column 2") +  # Label for the y-axis
  ggtitle("Scatter Plot of Numeric Column 1 vs Numeric Column 2")  # Title for the scatter plot
scatter_plot  # Display the scatter plot
```

The parameters used in the scatter plot include:

- `alpha`: Controls the transparency of the points; a value of 0.5 allows overlapping points to be seen more clearly.
- `aes(x, y)` Defines the mapping of variables to the x and y axes, allowing for easy identification of relationships.

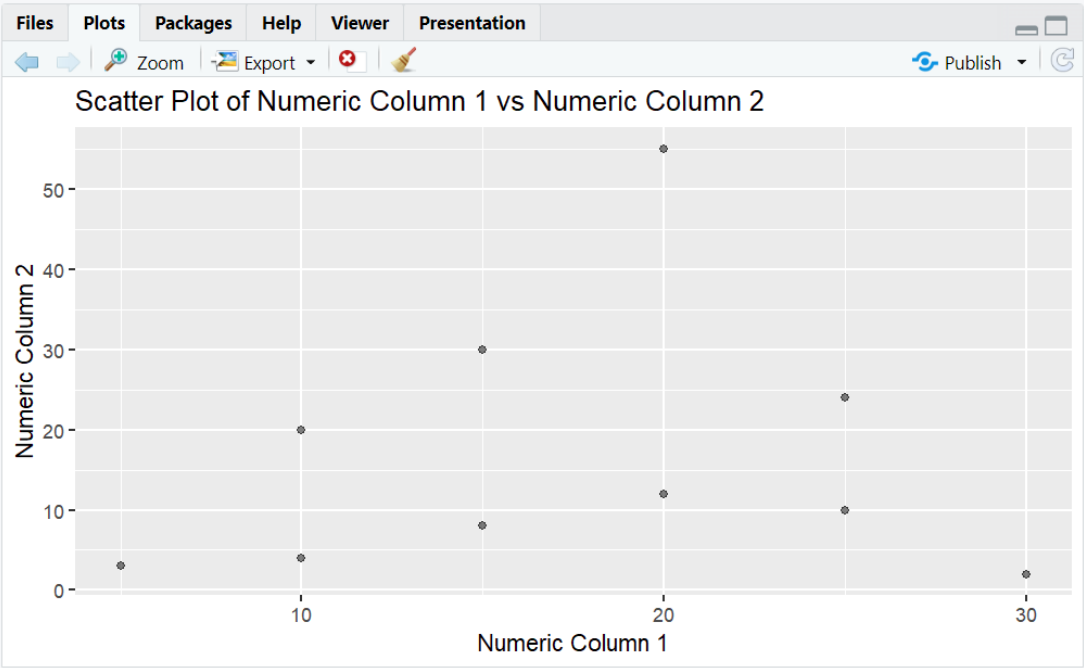

Figure 2.4: Output

Through these visualizations, we can gain valuable insights into the distribution of our data and the relationships between different numeric variables. Further analysis may involve creating additional plots, such as boxplots, to explore the data more thoroughly.

Missing Value Detection

Detecting missing values is a critical part of EDA. It helps in identifying incomplete data and taking appropriate actions, such as imputing missing values or removing affected rows or columns.

```r
# Check for missing values in each column
colSums(is.na(clean_data))  # Count missing values in each column
```

```
> colSums(is.na(clean_data))  # Count missing values in each column
   column1    column2 new_column
         0          0          0
>
```

Figure 2.5: Output

Correlation Analysis

Correlation analysis examines relationships between numeric variables, which can reveal redundancy or important associations. A correlation matrix provides a visual representation of these relationships.

```r
# Correlation matrix
correlation_matrix <- cor(clean_data %>% select_if(is.numeric), use =
↪  "complete.obs")

# Visualize correlation matrix as a heatmap
library(ggcorrplot)
ggcorrplot(correlation_matrix, method = "circle", lab = TRUE, title =
↪  "Correlation Matrix Heatmap")
```

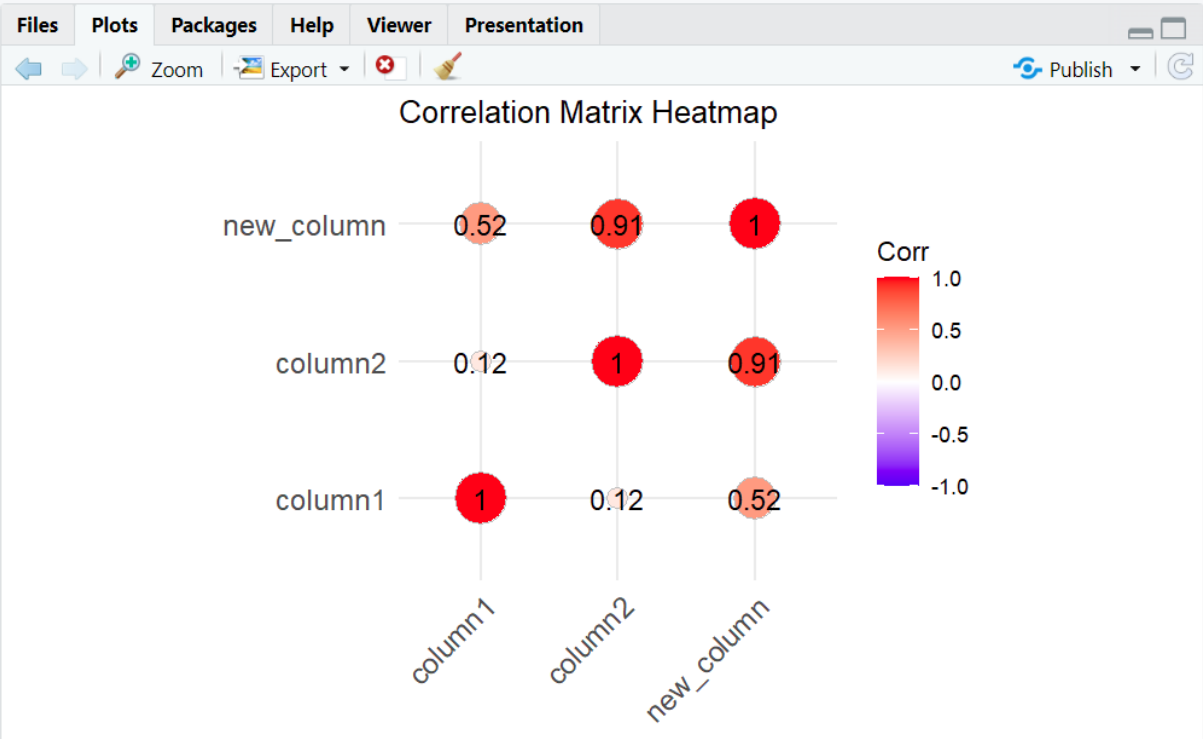

Figure 2.6: Output

These EDA techniques provide valuable insights into the data, assisting in identifying patterns, relationships, and potential data quality issues.

2.1.3 Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a dimensionality reduction technique commonly used to transform a large set of variables into a smaller set while retaining most of the original variability in the data. This technique is particularly useful in high-dimensional datasets, as it simplifies analysis and enables data visualization in a reduced space.

Performing PCA in R

In R, PCA can be performed using the `prcomp()` function, which efficiently computes principal components. It is important to standardize the data to ensure that each variable contributes equally to the analysis. Here, we will demonstrate PCA on the `iris` dataset.

```r
# Load the iris dataset and standardize the data
data <- iris[, -5]  # Exclude the Species column as it is categorical
scaled_data <- scale(data)  # Scale the data to have zero mean and unit
↪  variance
```

With the data standardized, we can proceed to perform PCA using the `prcomp()` function. This will help in determining the principal components and the amount of variance they explain.

```r
# Perform PCA on the standardized data
pca_result <- prcomp(scaled_data, center = TRUE, scale. = TRUE)  # Compute
↪  the principal components
```

To understand how much variance each principal component explains, we can summarize the PCA output.

```r
# Summarize PCA results
summary(pca_result)  # Display the proportion of variance explained by each
↪  principal component
```

```
Importance of components:
                          PC1    PC2     PC3     PC4
Standard deviation     1.7084 0.9560 0.38309 0.14393
Proportion of Variance 0.7296 0.2285 0.03669 0.00518
Cumulative Proportion  0.7296 0.9581 0.99482 1.00000
>
```

Figure 2.7: Output

The summary provides essential metrics, including the standard deviation, proportion of variance, and cumulative proportion of variance for each principal component. This helps determine the number of components that capture the majority of the variance in the data.

Visualizing PCA Results

A scree plot can help visualize the proportion of variance explained by each principal component, indicating the most significant components. We can use the base `screeplot()` function and also create a scree plot with `ggplot2` for a more customized view.

```r
# Base R scree plot
screeplot(pca_result, type = "lines", main = "Scree Plot")
```

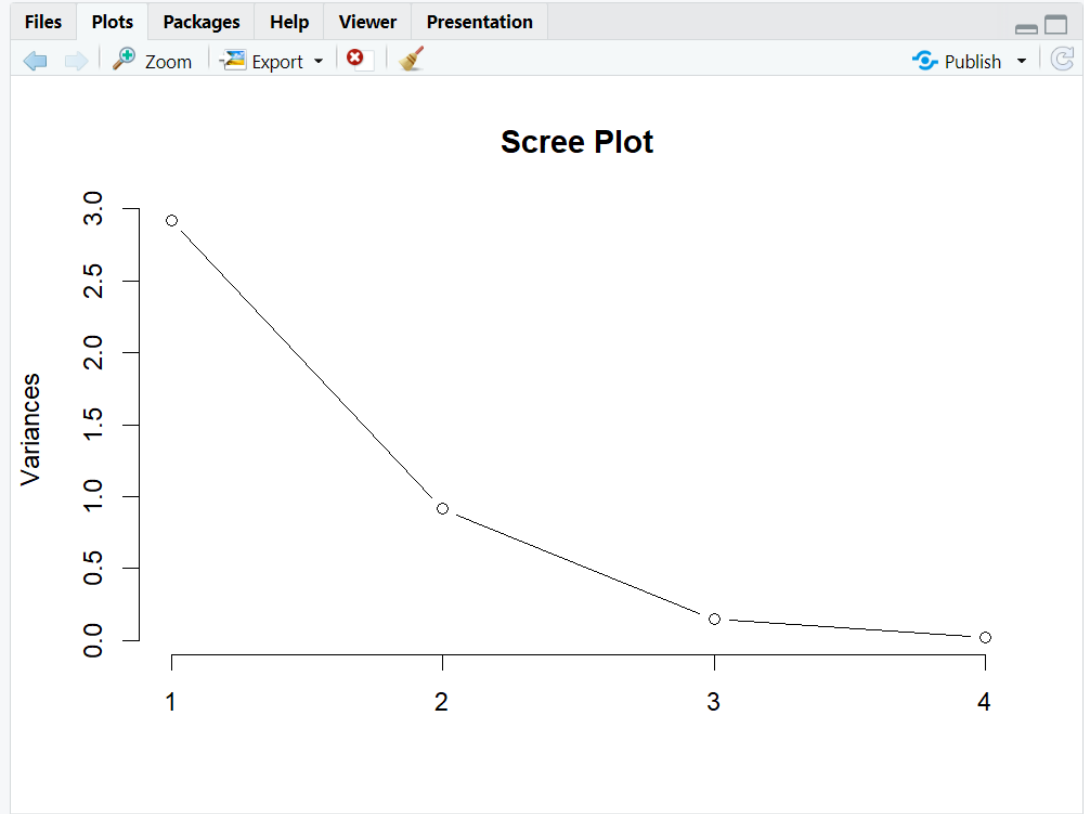

Figure 2.8: Output

To further explore the relationship between variables and observations, a biplot provides a graphical representation of both the principal components and the original variables.

```
# Create a biplot to visualize PCA results
biplot(pca_result, main = "PCA Biplot")  # Displays observations and variable relationships in PCA space
```

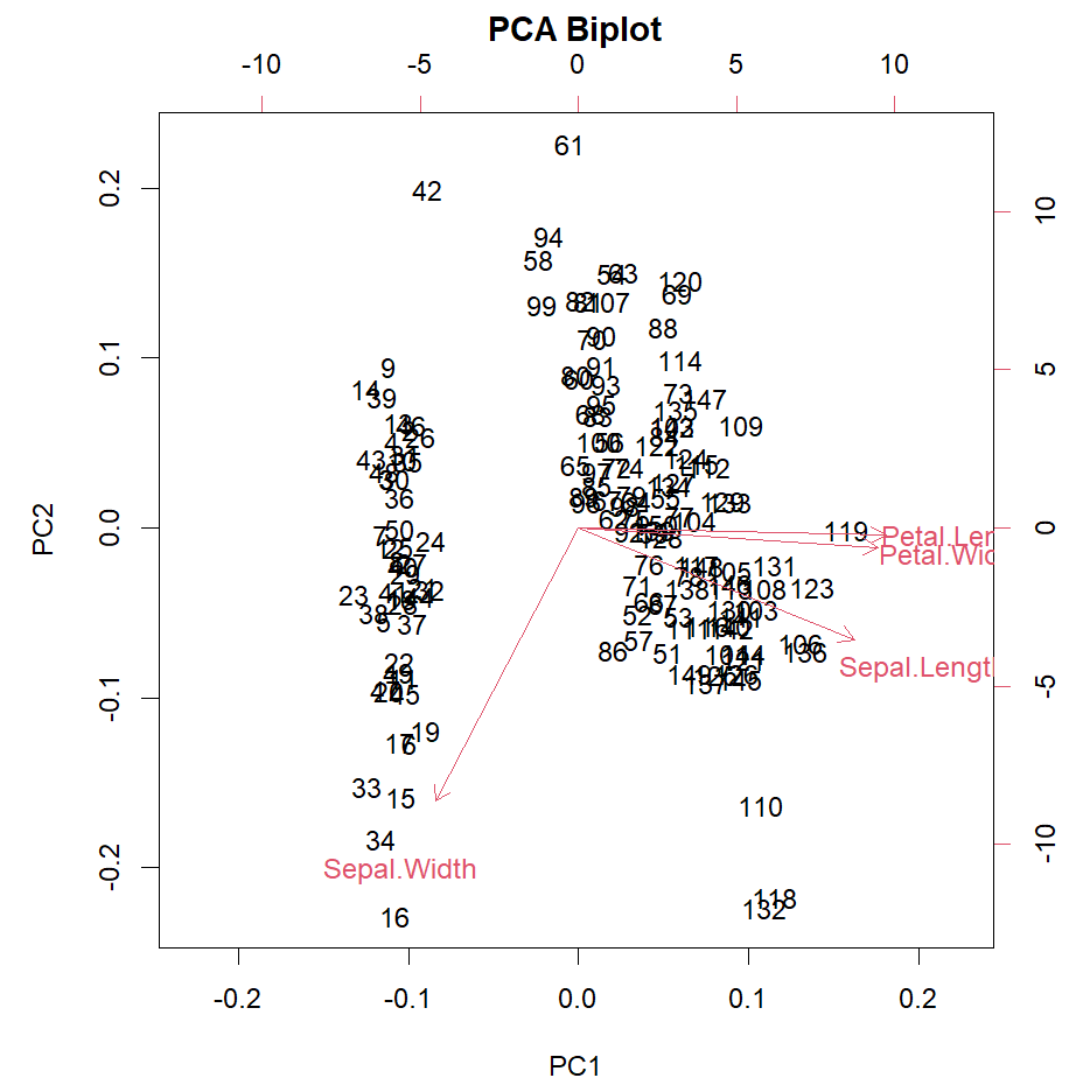

Figure 2.9: Output

2.2 Statistical Modeling

2.2.1 Linear Regression

In this section, we will fit a linear regression model to explore the relationship between a dependent variable and one independent variable. Linear regression is a statistical method used to model the relationship between one or more predictor variables and a response variable by fitting a linear equation to the observed data. This model helps us understand how changes in the independent variable impact the dependent variable.

Fitting a Linear Regression Model

To begin, we will fit a linear regression model using the `lm()` function in R, which allows us to specify the dependent and independent variables.

```r
# Fit a linear model
model <- lm(dependent_variable ~ independent_variable, data = clean_data)  # Create a linear model predicting 'dependent_variable' using 'independent_variable'
```

Next, we can summarize the fitted model to obtain statistical information about the coefficients, residuals, and overall model performance.

```r
# Summarize the model
summary(model)  # Display the summary statistics of the fitted model
```

```
> summary(model)

Call:
lm(formula = column1 ~ column2, data = clean_data)

Residuals:
     Min       1Q   Median       3Q      Max 
-11.7288  -5.8979  -0.8215   6.0153  13.3271 

Coefficients:
            Estimate Std. Error t value Pr(>|t|)   
(Intercept) 16.56110    3.87841    4.27  0.00272 **
column2      0.05589    0.16946    0.33  0.75003   
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 8.329 on 8 degrees of freedom
Multiple R-squared:  0.01341,	Adjusted R-squared:  -0.1099 
F-statistic: 0.1088 on 1 and 8 DF,  p-value: 0.75

> 
```

Figure 2.10: Output

The summary provides essential metrics, including the coefficients for the intercept and independent variable, R-squared value, and p-values, which help us assess the significance of the predictors.

2.2.2 Logistic Regression

In this section, we will implement a logistic regression model to predict a binary outcome based on multiple predictors. Logistic regression is a statistical method used for modeling the probability of a binary outcome based on one or more predictor variables. This model is essential for understanding how various factors influence a dichotomous outcome, such as success/failure, yes/no, or 1/0.

Fitting a Logistic Regression Model

To begin, we will fit a logistic regression model using the `glm()` function in R. We specify the binary outcome variable and the predictors, along with the family parameter set to `binomial`.

```r
# Fit a logistic regression model
logit_model <- glm(binary_outcome ~ predictor1 + predictor2,
                   data = clean_data, family = binomial)  # Fit a logistic
                   ↪  regression model for binary outcomes
```

Next, we will summarize the fitted logistic regression model to assess its performance and the significance of the predictors.

```r
# Summarize the model
summary(logit_model)  # Display the summary statistics of the fitted logistic
↪  model
```

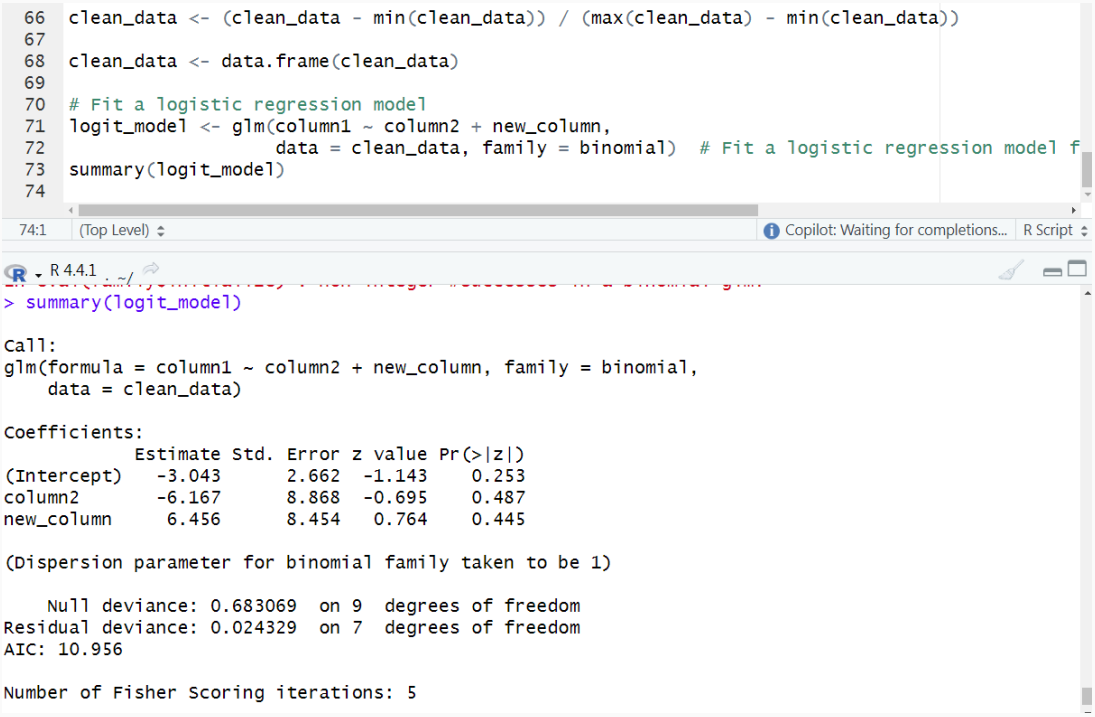

Figure 2.11: Output

The summary output includes key metrics such as the coefficients for each predictor, standard errors, z-values, and associated p-values.

2.3 Machine Learning

2.3.1 K-Nearest Neighbors (KNN)

In this section, we will apply the K-Nearest Neighbors algorithm, a simple yet effective method for classification tasks. KNN classifies a data point based on how its neighbors are classified, making it intuitive and easy to implement. We will split our data into training and testing sets before training the model to evaluate its performance.

Preparing the Data

To start, we need to load the necessary library:

```r
library(class)  % Load the class package for KNN
```

Next, we will define sample dataset. Suppose that we have two classes as follow:

```r
# Set seed for reproducibility
set.seed(42)

# Generate 100 data points for Class 1
class1 <- data.frame(
  feature1 = rnorm(100, mean = 2, sd = 1),
  feature2 = rnorm(100, mean = 3, sd = 1),
  class = factor(rep(1, 100))
)

# Generate 100 data points for Class 2
class2 <- data.frame(
  feature1 = rnorm(100, mean = 5, sd = 1),
  feature2 = rnorm(100, mean = 6, sd = 1),
  class = factor(rep(2, 100))
)
```

where `rnorm` is a command in `R` to generate a set of datapoints satisfying a given distribution, with mean `mean` and standard deviation `sd`.

Next, we use `rbind` to concatenate these two classes to form a single dataset:

```r
# Combine the data into a single dataset
sample_data <- rbind(class1, class2)
```

Now, let's divide the dataset into training set and test set:

```r
# Split data into training and testing sets
set.seed(123)
train_indices <- sample(1:nrow(sample_data), 0.7 * nrow(sample_data))
train_data <- sample_data[train_indices, ]
test_data <- sample_data[-train_indices, ]
```

Here, we have created a sample dataset with two classes and split it into training and testing sets. The first 70% of the data is used for training, and the remaining 30% is used for testing.

Also take out the features of training and testing set:

```r
# Separate features and labels
train_features <- train_data[, 1:2]
train_labels <- train_data$class
test_features <- test_data[, 1:2]
```

The features and labels are separated for both sets, enabling us to apply the KNN algorithm.

KNN Classification Visualization

In this section, we will visualize the KNN classification results using the decision boundaries, training points, and testing points. The decision boundaries represent the regions where the model predicts each class.

We start by creating a grid of values over the feature space, which will be used to visualize the decision boundaries for each class.

```r
# Create a grid of values over the feature space
grid <- expand.grid(
  feature1 = seq(min(sample_data$feature1) - 1, max(sample_data$feature1) + 1, by = 0.1),
  feature2 = seq(min(sample_data$feature2) - 1, max(sample_data$feature2) + 1, by = 0.1)
)

# Predict classifications for each point in the grid using the trained KNN model
grid$predicted <- knn(
  train = train_features,
  test = grid,
  cl = train_labels,
  k = k
)
```

In this code, we create a grid of values for both features using `expand.grid`. Then, we use the KNN model to predict the class for each point in the grid.

Plotting Decision Boundaries and Points

Now that we have the predicted values for the grid, we can plot the decision boundaries, training points, and testing points using `ggplot2`.

```r
# Plot the points and decision boundaries
ggplot() +
  # Plot decision boundaries using geom_tile()
  geom_tile(data = grid, aes(x = feature1, y = feature2, fill = predicted), alpha = 0.3) +
  # Plot training points
  geom_point(data = train_data, aes(x = feature1, y = feature2, color = class), shape = 19, size = 3, alpha = 0.8) +
  # Plot testing points
  geom_point(data = test_data, aes(x = feature1, y = feature2, color = class), shape = 1, size = 3, alpha = 0.8) +
  # Customizing plot aesthetics
  scale_fill_manual(values = c("lightblue", "lightgreen")) +
  scale_color_manual(values = c("blue", "green")) +
  labs(title = "KNN Classification and Decision Boundaries",
       x = "Feature 1", y = "Feature 2",
       fill = "Predicted Class", color = "Actual Class") +
  theme_minimal()
```

Here, we use:

- `geom_tile()` to fill grid cells based on the predicted class, which visualizes the decision boundaries.
- `geom_point()` to plot training and testing points with different shapes for easy distinction.
- `scale_fill_manual()` and `scale_color_manual()` to apply distinct colors to each class for clarity.

Figure 2.12: Output

Accuracy over the test set

```
# Evaluate the accuracy
accuracy <- mean(predicted_labels == test_data$class)
print(paste("Accuracy:", round(accuracy * 100, 2), "%"))
```

```
> accuracy <- mean(predicted_labels == test_data$class)
> print(paste("Accuracy:", round(accuracy * 100, 2), "%"))
[1] "Accuracy: 100 %"
>
```

Figure 2.13: Output

The above code applies KNN with $k = 5$ to classify the test data based on the training data. The accuracy is then calculated by comparing the predicted labels to the actual labels.

2.3.2 Random Forest

In this segment, we will train a Random Forest model. This ensemble learning method combines multiple decision trees to improve predictive accuracy and control overfitting. Random Forest is particularly useful for both classification and regression tasks due to its ability to handle a large number of input variables and to provide insights into feature importance.

Loading Required Libraries

First, we need to load the necessary library for Random Forest modeling:

```
library(randomForest)  # Load the randomForest package for Random Forest modeling
```

Training the Random Forest Model

Next, we will fit a Random Forest model using our training data. The model will use all other variables as predictors and will be configured to create a specified number of trees.

```
# Train a random forest model
rf_model <- randomForest(class ~ ., data = train_data, ntree = 100)  # Fit a Random Forest model with 100 trees using all other variables as predictors
```

In this example, `ntree = 100` indicates that the model will use 100 decision trees to make predictions. The `target ~` formula signifies that the target variable is being predicted using all other columns in the dataset.

Visualizing Feature Importance

To understand which features contribute most to the predictions made by the Random Forest model, we can visualize feature importance.

```r
# Load the randomForest package if not already loaded
library(randomForest)

# Calculate feature importance
importance_rf <- importance(rf_model)  # Get the importance of features from the Random Forest model

# Convert to a data frame for plotting
importance_df <- data.frame(Feature = rownames(importance_rf), Importance = importance_rf[, 1])

# Plot feature importance
library(ggplot2)  # Load ggplot2 for visualization
ggplot(importance_df, aes(x = reorder(Feature, Importance), y = Importance)) +
  geom_bar(stat = "identity", fill = "skyblue") +
  coord_flip() +
  xlab("Features") +
  ylab("Importance") +
  ggtitle("Feature Importance from Random Forest Model")
```

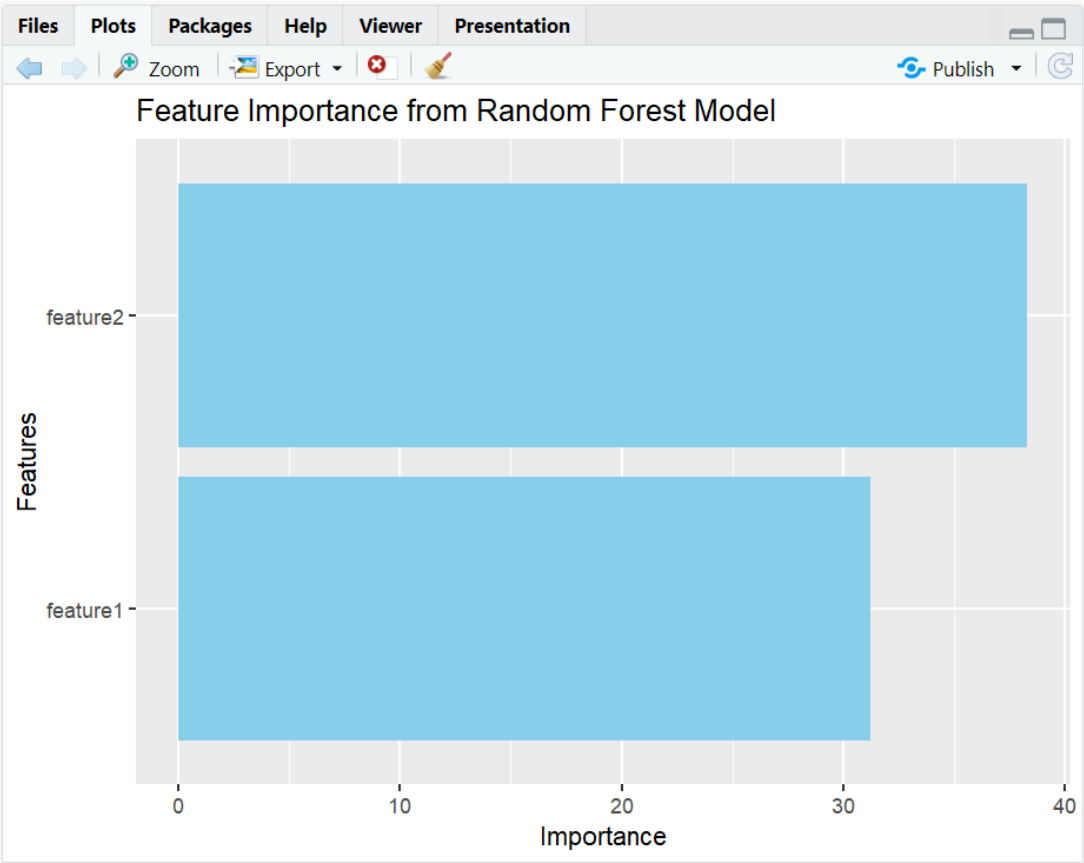

Figure 2.14: Output

Making Predictions and Evaluating Accuracy

After training the model, we can make predictions on the test data and evaluate its performance by calculating the accuracy.

```
# Predict and calculate accuracy
rf_pred <- predict(rf_model, test_data)  # Make predictions on the test data
accuracy_rf <- mean(rf_pred == test_data$class)  # Calculate the accuracy by
↪  comparing predictions to actual test labels
accuracy_rf  # Display the accuracy
```

```
> accuracy_rf <- mean(rf_pred == test_data$class)  # Calculate the accuracy b
y comparing predictions to actual test labels
> accuracy_rf  # Display the accuracy
[1] 0.9833333
>
```

Figure 2.15: Output

The accuracy metric provides a straightforward measure of how well our Random Forest model is performing on the unseen test data.

2.4 Model Evaluation

2.4.1 Confusion Matrix and Accuracy

To assess the performance of our classification models, we will generate a confusion matrix. A confusion matrix is a useful tool for evaluating the performance of a classification model by summarizing the prediction results. It displays the counts of true positive, true negative, false positive, and false negative predictions. This allows us to understand not only the overall accuracy of the model but also the types of errors it makes, enabling more informed decisions for improving the model.

The key components of a confusion matrix are:

- `True Positives (TP)`: The count of instances correctly predicted as the positive class.
- `True Negatives (TN)`: The count of instances correctly predicted as the negative class.
- `False Positives (FP)`: The count of instances incorrectly predicted as the positive class (also known as Type I error).
- `False Negatives (FN)`: The count of instances incorrectly predicted as the negative class (also known as Type II error).

These components provide a foundation for calculating additional performance metrics, such as accuracy, sensitivity (recall), specificity, and the F1 score, which are crucial for understanding the model's effectiveness.

Loading Required Libraries

First, we need to load the necessary library for model evaluation:

```r
library(caret)  # Load the caret package for model evaluation
```

Generating the Confusion Matrix

Next, we will generate a confusion matrix comparing the predicted labels from our Random Forest model with the actual labels in the test data.

```r
# Confusion matrix
confusion_mat <- confusionMatrix(data = rf_pred, reference = test_data$class)
↪  # Generate a confusion matrix comparing predictions to actual values
confusion_mat  # Display the confusion matrix
```

```
> confusion_mat  # Display the confusion matrix
Confusion Matrix and Statistics

          Reference
Prediction  1  2
         1 28  0
         2  1 31

               Accuracy : 0.9833
                 95% CI : (0.9106, 0.9996)
    No Information Rate : 0.5167
    P-Value [Acc > NIR] : 3.544e-16

                  Kappa : 0.9666

 Mcnemar's Test P-Value : 1

            Sensitivity : 0.9655
            Specificity : 1.0000
         Pos Pred Value : 1.0000
         Neg Pred Value : 0.9688
             Prevalence : 0.4833
         Detection Rate : 0.4667
   Detection Prevalence : 0.4667
      Balanced Accuracy : 0.9828

       'Positive' Class : 1

>
```

Figure 2.16: Output

The confusion matrix will provide us with key metrics such as accuracy, sensitivity, specificity, and other relevant statistics that help evaluate our model's performance.

2.4.2 ROC Curve

In this section, we will create a Receiver Operating Characteristic (ROC) curve to evaluate the performance of our classification model. The ROC curve illustrates the trade-off between the true positive rate (sensitivity) and false positive rate (1 - specificity) at various threshold settings.

Loading the ROC Analysis Library

First, we need to load the `pROC` library, which provides tools for ROC analysis:

```
library(pROC)  # Load the pROC package for ROC analysis
```

Creating the ROC Curve

We will create an ROC object using the actual outcomes and predicted probabilities, then plot the ROC curve to visualize the model's performance:

```
# ROC Curve
roc_obj <- roc(test_data$class, as.numeric(rf_pred))  # Create an ROC object using actual and predicted values
plot(roc_obj, col = "blue", main = "ROC Curve")  # Plot the ROC curve in blue
```

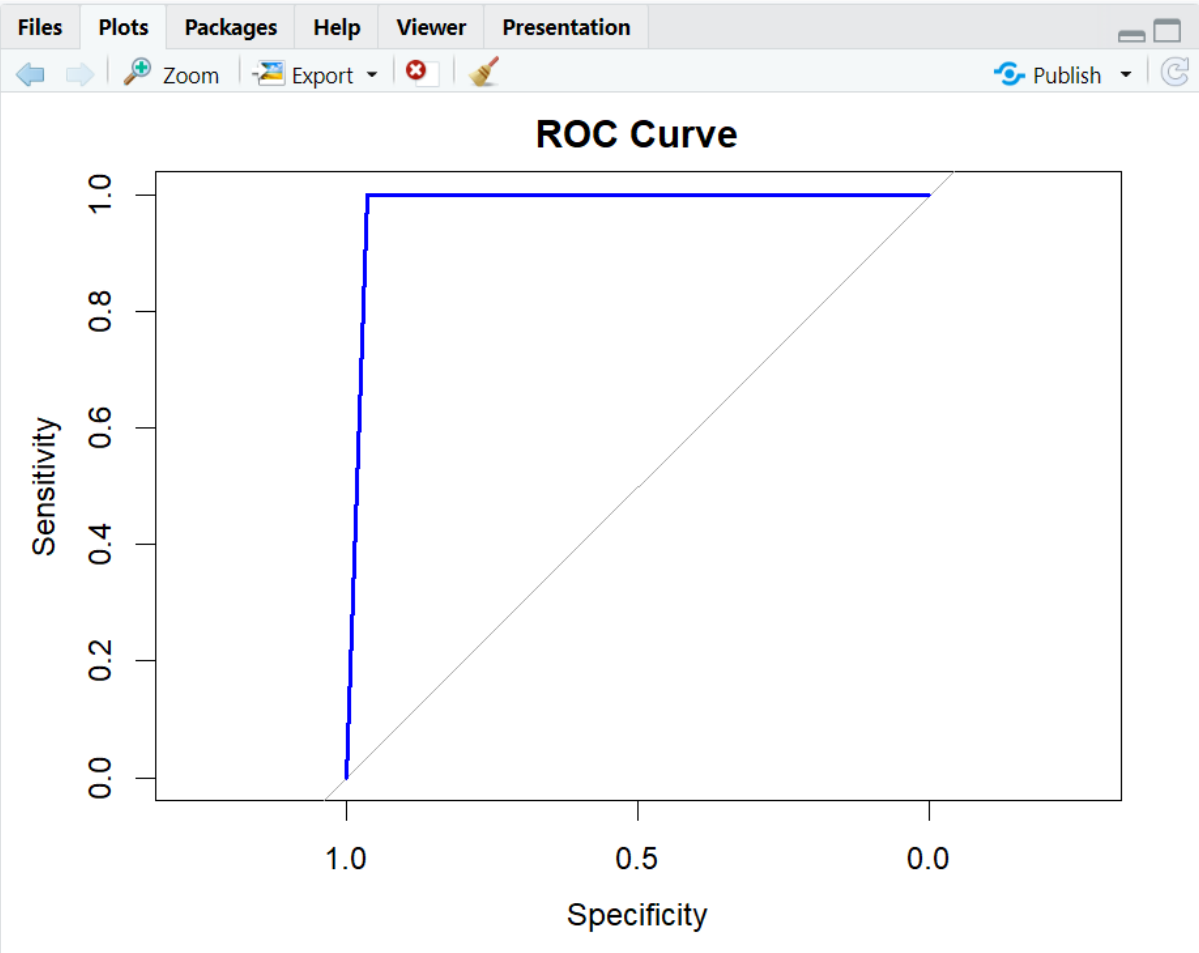

Figure 2.17: Output

The ROC curve will allow us to assess the model's discriminative ability. The area under the ROC curve (AUC) is a common metric to summarize the overall performance

of the classifier. AUC values range from 0 to 1, where a value of 0.5 indicates no discriminative ability and a value of 1 indicates perfect discrimination.

Chapter 3

Deep Learning Features in R

R has emerged as a versatile tool for deep learning, offering a range of packages that enable users to design, train, and deploy neural networks.

3.1 Popular Packages for Deep Learning

R provides several packages that facilitate the implementation of deep learning models. Among the most widely used packages are:

- `keras`: A high-level neural networks API that allows users to build and train models using a simple and intuitive syntax. The `keras` package in R is a wrapper for the Keras library in Python, making it easy to deploy TensorFlow-powered deep learning models.
- `tensorflow`: The `tensorflow` package provides an interface to Google's TensorFlow library, which is widely used for building deep learning models. This package allows users to construct and customize models directly using TensorFlow's low-level API.

3.2 Building Neural Networks with Keras

The `keras` package simplifies the process of building neural networks in R. Below, we demonstrate how to build a feedforward neural network for classifying images from the MNIST dataset. We start by loading and preprocessing the data, then define and compile the model.

3.2.1 Loading and Preprocessing Data

To begin, we load the MNIST dataset, split it into training and testing sets, and prepare it for model input.

```r
library(keras)

# Load the MNIST dataset
mnist <- dataset_mnist()
```

The code above loads the `keras` library and retrieves the MNIST dataset using `dataset_mnist()`, which returns a list containing the training and test sets.

```r
# Split data into training and test sets
c(x_train, y_train) %<-% mnist$train
c(x_test, y_test) %<-% mnist$test
```

The training and test sets are extracted from the `mnist` list and assigned to separate variables for images (`x_train`, `x_test`) and labels (`y_train`, `y_test`).

```r
# Reshape the images to 1D vectors (28x28 = 784)
x_train <- array_reshape(x_train, c(nrow(x_train), 784))
x_test <- array_reshape(x_test, c(nrow(x_test), 784))
```

Each 28×28 pixel image is reshaped into a single-dimensional array of 784 values. This is necessary for input into the neural network, which expects one-dimensional input vectors.

```r
# Normalize the pixel values to the range [0, 1]
x_train <- x_train / 255
x_test <- x_test / 255
```

The pixel values are scaled to the range $[0, 1]$ by dividing by 255. This standardization improves the training process, making it more efficient.

```r
# One-hot encode the labels
y_train <- to_categorical(y_train, 10)
y_test <- to_categorical(y_test, 10)
```

The labels are then converted to one-hot encoded vectors using `to_categorical`. This step is crucial for categorical classification tasks, as it represents each digit as a 10-dimensional vector, where each position corresponds to a class.

3.2.2 Defining the Model

After preprocessing, we define the neural network model as follows:

```r
# Initialize a sequential model
model <- keras_model_sequential() %>%
  # First dense layer with 128 units and ReLU activation
  layer_dense(units = 128, activation = "relu", input_shape = c(784)) %>%
  # First dropout layer with a 40% drop rate
  layer_dropout(rate = 0.4) %>%
  # Second dense layer with 64 units and ReLU activation
  layer_dense(units = 64, activation = "relu") %>%
  # Second dropout layer with a 30% drop rate
  layer_dropout(rate = 0.3) %>%
  # Output layer with 10 units and softmax activation for classification
  layer_dense(units = 10, activation = "softmax")
```

The model consists of three dense layers interleaved with dropout layers to prevent overfitting. The final layer applies the softmax function to produce class probabilities for each digit.

3.2.3 Compiling the Model

Finally, we compile the model with the Adam optimizer and the categorical cross-entropy loss function:

```
# Compile the model with optimizer, loss function, and metric
model %>% compile(
  optimizer = "adam",
  loss = "categorical_crossentropy",
  metrics = c("accuracy")
)
```

The Adam optimizer dynamically adjusts learning rates during training, while the categorical cross-entropy loss function is suited for multi-class classification. The model will also report accuracy after each epoch.

3.3 Training Deep Learning Models

Once the model is constructed, it can be trained using the `fit()` function, which takes in the training data, labels, and additional parameters such as batch size and number of epochs.

```
# Train the model
history <- model %>% fit(
  x_train, y_train,
  epochs = 20,
  batch_size = 128,
  validation_split = 0.2
)
```

The training history can be visualized to assess the model's performance over epochs.

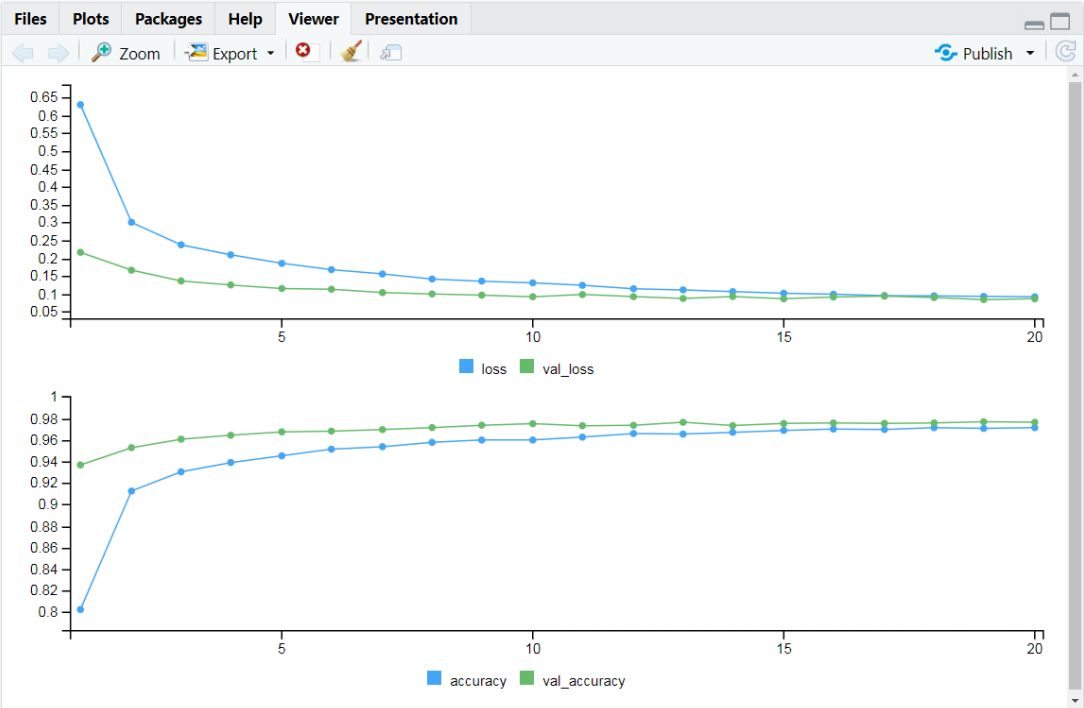

Figure 3.1: Training curve

We can also see the training progress in the terminal:

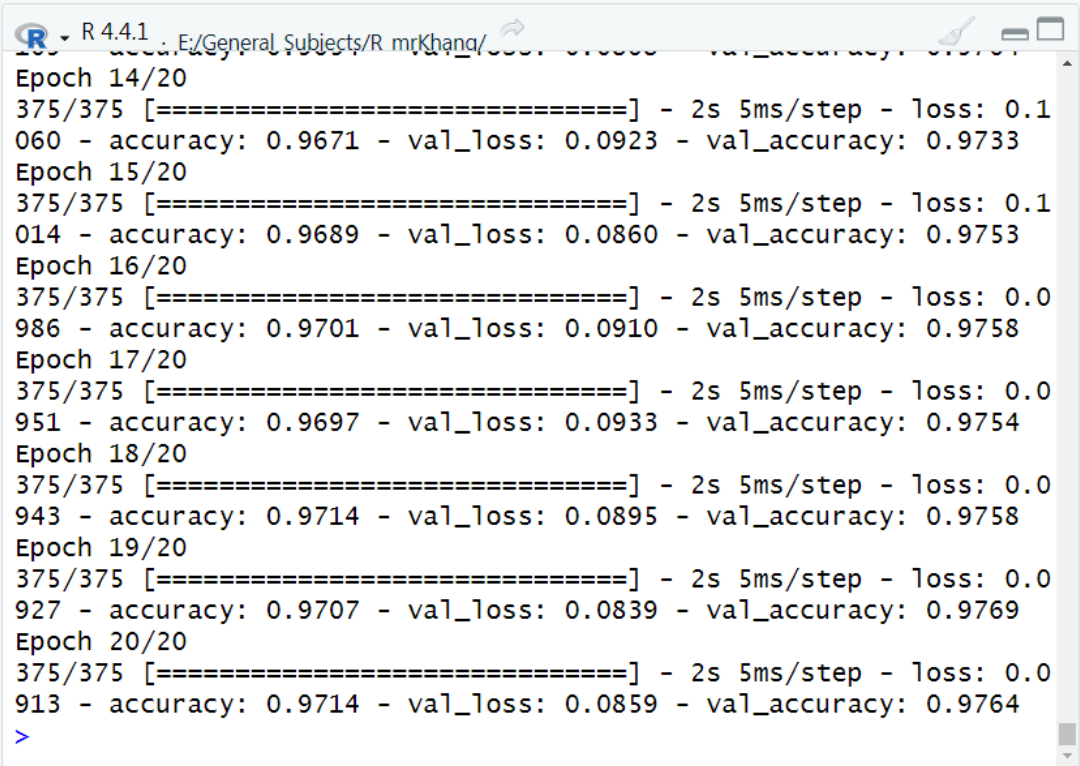

Figure 3.2: Training loss

Now, let's evaluate our model. We can simply use `evaluate()` to check the accuracy. The return value will be a list, with first element being the loss and second element being

the accuracy of the model:

```
# Evaluate the model
score <- model %>% evaluate(x_test, y_test, verbose = 0)
cat('Test loss:', score[1], "\n")
cat('Test accuracy:', score[2], "\n")
```

```
> score <- model %>% evaluate(x_test, y_test, verbose = 0)
> cat('Test loss:', score[1], "\n")
Test loss: 0.07799443
> cat('Test accuracy:', score[2], "\n")
Test accuracy: 0.9765
>
```

Figure 3.3: Output

The accuracy is 0.9765. Not bad!

```
# Plot training & validation accuracy values
plot(history)
```

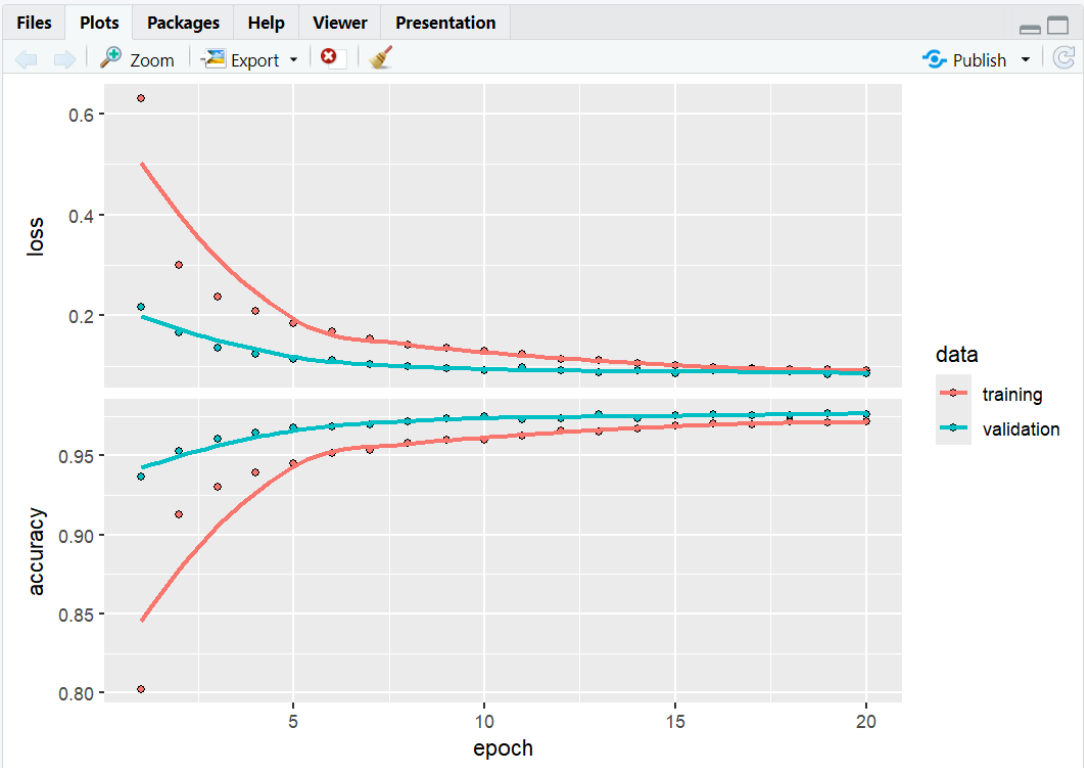

Figure 3.4: Output

There we go!

3.4 Using Pretrained Models with the CIFAR Dataset

The `keras` package provides access to several pretrained models, including VGG16, ResNet, and Inception. In this example, we will fine-tune the VGG16 model on the CIFAR-10 dataset. CIFAR-10 is a widely used dataset in machine learning, containing 60,000 32x32 color images across 10 different classes, with 6,000 images per class. The dataset is split into 50,000 training images and 10,000 test images.

We will break down the code into smaller sections and explain each part.

3.4.1 Load and Preprocess the CIFAR-10 Dataset

First, we load the CIFAR-10 dataset, separate the training and testing data, and preprocess the images. This involves resizing them to 224×224 pixels and normalizing the pixel values to the range $[0, 1]$.

```r
# Load the CIFAR-10 dataset
cifar10 <- dataset_cifar10()
```

```
> # Load the CIFAR-10 dataset
> cifar10 <- dataset_cifar10()
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
150609920/170498071 [==========================>....] - ETA: 9s
```

Figure 3.5: Output

```r
# Extract training and test data
x_train <- cifar10$train$x
y_train <- cifar10$train$y
x_test <- cifar10$test$x
y_test <- cifar10$test$y

# Normalize the pixel values to be in the range [0, 1]
x_train <- x_train / 255
x_test <- x_test / 255

# Convert labels to categorical (one-hot encoding)
y_train <- to_categorical(y_train, num_classes = 10)
y_test <- to_categorical(y_test, num_classes = 10)
```

The images are resized to meet the input requirements of VGG16, and the pixel values are normalized for better convergence during training. Labels are also converted to categorical format for multi-class classification.

3.4.2 Load and Modify the Pretrained ResNet50 Model

Next, we load the ResNet50 model with pretrained ImageNet weights and remove the top classification layer, allowing us to add our own custom layers.

```r
# Load the ResNet50 model without the top layer
pretrained_model <- application_resnet50(weights = "imagenet", include_top =
↪  FALSE, input_shape = c(32, 32, 3))
```

```
> pretrained_model <- application_vgg16(weights = "imagenet", include_top = FALSE, input_sha
WARNING:tensorflow:From C:\Users\HP\DOCUME~1\VIRTUA~2\R-TENS~1\Lib\site-packages\keras\src\

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg
58889256/58889256 [==============================] - 22s 0us/step
```

Figure 3.6: Output

Setting `include_top = FALSE` removes the dense layers at the top of the ResNet50 model, enabling us to add new layers tailored for the CIFAR-10 classification task.

3.4.3 Add Custom Layers for Fine-Tuning

We add a flattening layer, a dense layer with ReLU activation, a dropout layer for regularization, and a final dense layer with softmax activation for classification.

```r
# Add custom layers for fine-tuning
custom_model <- pretrained_model$output %>%
  layer_flatten() %>%
  layer_dense(units = 256, activation = "relu") %>%
  layer_dropout(rate = 0.5) %>%
  layer_dense(units = 10, activation = "softmax")

# Create a new model combining the pretrained base and the custom layers
model <- keras_model(inputs = pretrained_model$input, outputs = custom_model)
```

The new layers will classify the CIFAR-10 images. The dropout layer reduces overfitting, and the final layer uses softmax for multi-class classification.

3.4.4 Compile the Model

We compile the model, specifying the `adam` optimizer, `categorical_crossentropy` loss function, and `accuracy` as a metric.

```r
# Compile the model
model %>% compile(
  optimizer = "adam",
  loss = "categorical_crossentropy",
  metrics = c("accuracy")
)
```

This setup is standard for multi-class classification problems, where accuracy is the evaluation metric.

3.4.5 Train the Model

We train the model on the training data, using 20% of the data for validation, and specify the number of epochs and batch size.

```
# Train the model
model %>% fit(
  x_train, y_train,
  epochs = 5,
  batch_size = 32,
  validation_split = 0.2
)
```

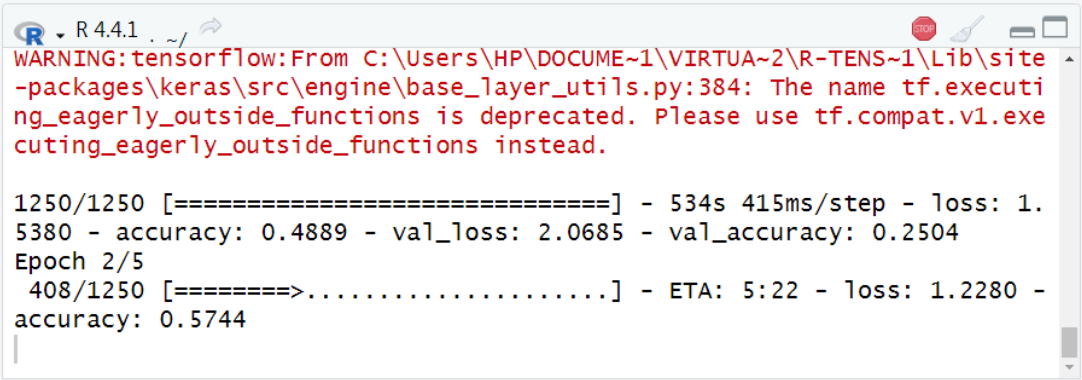

Figure 3.7: Training progress

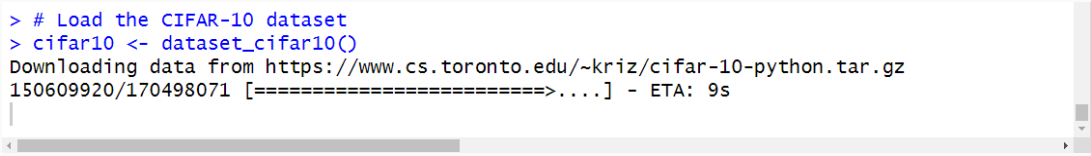

Figure 3.8: Training curve

Here, `validation_split = 0.2` sets aside 20% of the training data for validation, helping us monitor the model's performance during training.

3.4.6 Evaluate the Model on Test Data

Finally, we evaluate the trained model on the test data to assess its accuracy.

```
# Evaluate the model on the test set
score <- model %>% evaluate(x_test, y_test, verbose = 0)
cat('Test loss:', score[1], "\n")
cat('Test accuracy:', score[2], "\n")
```

```
> # Evaluate the model on test data
> score <- model %>% evaluate(x_test, y_test, verbose = 0)
> cat('Test loss:', score[1], "\n")
Test loss: 1.634395
> cat('Test accuracy:', score[2], "\n")
Test accuracy: 0.4563
>
```

Figure 3.9: Output

This code prints the model's loss and accuracy on the test set, providing an evaluation of how well it generalizes to unseen data.